Path Planning of Low-Altitude UAV for Tree Canopy Tracking and Orchard Monitoring

Kaushal Kishore
CSIR–Central Electronics Engineering Research Institute (CSIR-CEERI)
Pilani, Rajasthan, India
kaushal@ceeri.res.in

Ritabrata Chakraborty
Birla Institute of Technology and Science (BITS) Pilani
Pilani, Rajasthan, India
f20220843@pilani.bits-pilani.ac.in

***Abstract*—Orchards are planted in long parallel rows, so a monitoring UAV must lock onto one treeline and follow it without drifting to a neighboring row. This paper presents a monocular, onboard pipeline that does this from canopy appearance alone. A forward-facing camera tilted 45° downward feeds a YOLOv11 instance-segmentation model trained on a new olive-orchard dataset (290 annotated images captured in Pilani, India). Gaussian Soft-NMS keeps adjacent treelines as separate instances where they converge near the image edges, a Kalman filter keeps the selected row locked across frames, and a line fitted through the center points of the refined canopy mask gives the navigation line. Its horizontal offset and slope set the roll and yaw commands, while pitch is held constant. On the custom dataset, YOLOv11 reaches a box mAP@50 of 95.5% and a mask mAP@50 of 96.5%, ahead of YolactEdge and YOLACT++. With TensorRT optimization the pipeline runs at 17 frames per second on an NVIDIA Jetson Orin NX. In field tests over the Pilani olive orchard, the quadcopter generated and followed navigation lines autonomously and remained stable under small wind disturbances.**

***Index Terms*—Precision agriculture, unmanned aerial vehicles (UAVs), orchard monitoring, treeline tracking, instance segmentation, YOLOv11, Soft-NMS, Kalman filter, visual servoing, edge computing.**

I. Introduction

AUTONOMOUS UAVs are becoming routine tools in precision agriculture, where they support crop monitoring, resource optimization, and automated decision making. In orchards, row-by-row inspection underpins yield prediction, disease identification, and targeted input application. Compared with ground robots, UAVs cross difficult terrain easily and fly above the ground-level occlusions caused by shadows and obstacles.

Following a single treeline from the air is harder than it looks. The UAV must recognize the row it is assigned to, tell it apart from neighboring rows that look almost identical, and hold its course when wind pushes it sideways. Existing approaches handle only parts of this problem. Grid-based coverage planners assume an accurate map and do not perceive the rows themselves. Trunk-based vision fails when dense canopies or shadows hide the trunks. Semantic segmentation of the inter-row path breaks into disconnected pieces wherever gaps open between trees. A further, rarely discussed failure occurs inside the detector: near the image edges, where rows appear to converge, standard non-maximum suppression (NMS) merges adjacent treelines into a single detection.

This paper addresses these issues with a canopy-centric pipeline that runs entirely onboard (Fig. 1). The UAV flies above the canopy, segments each treeline as a separate instance, keeps the target row locked with a Kalman filter, and converts the row's geometry into roll and yaw commands. The main contributions are:

1) an olive-orchard dataset captured by UAV in Pilani, Rajasthan, India, with manual treeline annotations that support both object detection and instance segmentation;
2) an overlap-aware perception front end that combines YOLOv11 instance segmentation with Gaussian Soft-NMS and Kalman tracking, so converging treelines remain separable and the selected row stays locked;
3) a comparison of four navigation-line estimators (two straight-line fits and two B-spline fits) and a control law that maps the selected line's offset and slope to roll and yaw commands at constant pitch;
4) a real-time implementation on an NVIDIA Jetson Orin NX, benchmarked against YolactEdge and YOLACT++ and tested in field flights.

Section II reviews related work. Section III states the problem and gives a system overview, and Section IV describes the dataset. Section V details the perception and control pipeline, Section VI reports the experiments, Section VII discusses limitations, and Section VIII concludes.

II. Related Work

Research on autonomous agricultural navigation spans path planning, classical machine vision, and deep learning. Table I summarizes representative work in each area.

A. Coverage and Path Planning

Planning-centric methods optimize where a UAV should fly, assuming the field geometry is known. They include learned coverage planners [2], sprinkler-aware spray coverage [3], genetic algorithms for heterogeneous fleets [5], and hybrid A*/gravitational search [6]; Basiri *et al.* [4] survey this family for multirotor UAVs. UAV imagery is also used to assess orchard interventions such as pruning [1]. Planning methods of this kind rely on a map rather than on perceiving the rows, so they cannot correct for row-level drift during flight.

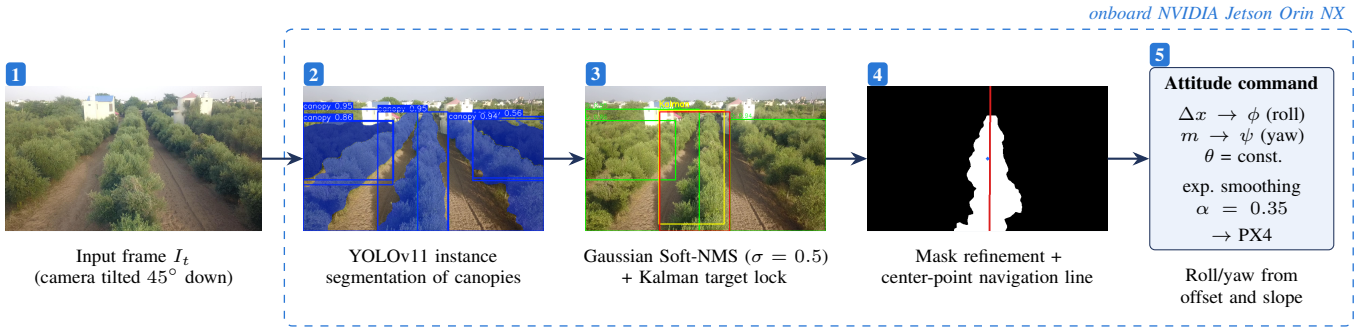


Fig. 1. Overview of the proposed treeline-tracking pipeline. Each frame is segmented into canopy instances (2), filtered with Gaussian Soft-NMS and associated with the tracked row by a Kalman filter (3), reduced to a single cleaned mask from which the navigation line is fitted (4), and converted to roll and yaw commands at constant pitch (5). The thumbnails are illustrative examples from the dataset and the experiments, not consecutive stages of one frame.

B. Classical Machine Vision

Vision-based guidance closes the loop on what the vehicle sees. Examples include sky-segmentation guidance between canopies [7], trunk-based directrix generation [8], least-squares path fitting for harvest robots [9], color–texture tree detection [10], and line following for UAVs [11]. Stefas *et al.* [12] built a complete aerial orchard-monitoring system, and Blok *et al.* [13] showed that the choice of state estimator affects localization quality in orchards.

C. Deep Learning

Learned perception now dominates [14]. Zhou *et al.* [15] detect trunks with YOLOv3 and derive the navigation line from them, Han *et al.* [16] segment the drivable path with U-Net, and trail-following networks [17], [18] map camera images directly to navigation decisions.

D. Gap Addressed by This Work

Three limitations recur across these methods. First, platforms operating below the canopy suffer occlusion and collision risk. Second, trunk-based methods [15] fail when foliage or shadows hide the trunks. Third, path segmentation fragments wherever gaps open between trees, a problem made worse in young orchards with open layouts. Flying above the canopy and segmenting the canopy itself avoids all three. It does, however, expose a new failure: converging rows suppress one another inside the detector. The overlap-aware front end in Section V is designed for this problem.

III. Problem Statement and System Overview

A. Problem Statement

A quadcopter flies above an orchard with a forward-facing camera tilted 45° downward. At time t the camera returns an image I_t of width W, whose center column is $x_{\text{img}} = W/2$. One treeline is designated as the target. The goal is to compute attitude commands that keep the UAV aligned with the target row: a yaw angle ψ that corrects the heading, a roll angle ϕ that corrects the lateral offset, and a constant pitch θ that sets the forward motion.

The commands are derived from a *navigation line* $\mathcal{L}$ fitted to the target treeline in image space. When the UAV is perfectly aligned with the row, $\mathcal{L}$ is vertical and passes through the image center, so the x-coordinates of its top and bottom points both equal $W/2$. Any horizontal displacement of $\mathcal{L}$ is corrected with roll, and any inclination with yaw.

B. System Overview

Fig. 1 shows the five stages of the pipeline, all of which run onboard on an NVIDIA Jetson Orin NX: (1) image acquisition; (2) canopy instance segmentation with YOLOv11; (3) overlap-aware filtering with Gaussian Soft-NMS and Kalman-based target association; (4) mask refinement and navigation-line fitting; and (5) the control law, whose smoothed roll, yaw, and constant-pitch commands are sent to the PX4 autopilot. Table II lists every parameter of the pipeline.

IV. Olive-Orchard Treeline Dataset

The dataset was built from video recorded by a quadcopter with a forward-facing DJI camera, flown along different treelines over an olive orchard in Pilani, Rajasthan, India. The videos were recorded at 1920×1080 pixels. Frames were extracted from the videos, and the treelines were annotated manually as ground truth, so the labels serve both object detection and instance segmentation. Fig. 2 shows representative frames, and Table III summarizes the dataset and the augmentations applied during training to improve robustness.

V. Methodology

A. Canopy Instance Segmentation

YOLOv11 [19], the latest member of the YOLO (You Only Look Once) family, predicts for each frame a set of N instances $\{\mathcal{B}_i, \mathcal{S}_i, \mathcal{M}_i\}_{i=1}^{N}$, each consisting of a bounding box, a confidence score, and a pixel-level mask. Because every treeline becomes a separate instance, adjacent rows can be told apart, which a semantic (per-class) segmentation cannot do. The model was trained on the dataset of Section IV with input images resized to 640×384 pixels.

B. Overlap-Aware Duplicate Removal

YOLO removes redundant boxes for the same object with non-maximum suppression (NMS), keeping only the highest-confidence detections. In orchards, this step is risky. Near the

TABLE I
REPRESENTATIVE WORK ON UAV AND ROBOT NAVIGATION IN AGRICULTURE

Category	Work	Approach	Reported outcome
Coverage and path planning	Roma *et al.* [1]	Multispectral UAV imagery to assess biennial pruning of olive trees	$R^2 = 0.97$ (canopy area), 0.96 (volume)
	Li *et al.* [2]	Double Deep Q-Network (DDQN) coverage path planning	Less path redundancy
	Vazquez-Carmona *et al.* [3]	Spraying-drone coverage planning with a realistic sprinkler model	More uniform spray distribution
	Basiri *et al.* [4]	Survey of path-planning algorithms for multirotor UAVs in precision agriculture	–
	Mukhamediev *et al.* [5]	Genetic-algorithm coverage planning for heterogeneous UAV groups	10% lower coverage cost
	Tian *et al.* [6]	Improved A* combined with gravitational search	-23.18% non-spraying distance; -64.47% pesticide overuse
Classical machine vision	Radcliffe *et al.* [7]	Sky-based machine vision for orchard navigation	RMS error 2.13 cm
	Feng *et al.* [8]	Trunk detection by image processing to generate a navigation directrix	$> 90.7\%$ accuracy
	He *et al.* [9]	Least-squares line fitting for harvest-robot path recognition	91.7% recognition rate
	Ali *et al.* [10]	Monocular tree detection from color and texture features	Distance estimation to trees
	Brandão *et al.* [11]	Vision-based line following for a UAV with a nonlinear controller	Stabilized crop-line tracking
	Stefas *et al.* [12]	Autonomous aerial orchard monitoring with vision-based obstacle avoidance	Orchard yield estimation
	Blok *et al.* [13]	Particle vs. Kalman filter localization of an orchard robot	Particle filter performed better
Deep learning	Popescu *et al.* [14]	Systematic review of UAV orchard monitoring with neural networks	–
	Zhou *et al.* [15]	YOLOv3 trunk detection for orchard navigation lines	92.11% mAP
	Han *et al.* [16]	U-Net segmentation of the orchard path	IoU 89.52% / 86.16% (weak / strong light)
	Back *et al.* [17]	CNN-based trail following with obstacle avoidance	Real-time UAV trail navigation
	Smolyanskiy *et al.* [18]	TrailNet DNN with entropy-based loss to reduce oscillations	Trail navigation in unstructured terrain
This work	–	YOLOv11 canopy instance segmentation, Gaussian Soft-NMS, Kalman lock, center-point line fit, above-canopy flight	Mask mAP@50 96.5%; 17 FPS on Jetson Orin NX

TABLE II
PIPELINE PARAMETERS

Stage	Parameter	Value
Camera	Mounting	Forward-facing, 45° downward
Segmentation	Model / input	YOLOv11, 640×384 px
Duplicate removal	Method	Gaussian Soft-NMS, $\sigma = 0.5$
Target tracking	Fusion weights	0.65 (Kalman estimate), 0.25 (detection)
Mask refinement	Opening kernel	5×5, elliptical
	Region kept	Largest connected component
Navigation line	Estimator	Straight line through center points
	Spline variants	Cubic and quadratic, smoothness 500
Control	Smoothing factor	$\alpha = 0.35$ (line, roll, yaw)
	Pitch	Held constant
Deployment	Hardware	NVIDIA Jetson Orin NX, TensorRT



Fig. 2. Representative frames from the olive-orchard dataset, captured during UAV flights in Pilani, Rajasthan, showing different treeline configurations and lighting conditions.

image edges the rows appear to converge, and their boxes overlap heavily. When a box that spans several treelines has the highest score, greedy NMS removes the boxes of the individual rows, and some treelines are missed altogether (Fig. 3b). Segmentation models such as U-Net suffer the same limitation: closely spaced treelines become indistinguishable.

Several refinements of NMS have been proposed:

- *Soft-NMS* [20] lowers the scores of overlapping proposals instead of discarding them (Algorithm 1). Like greedy

TABLE III
DATASET SUMMARY

Property	Value
Site	Olive orchard, Pilani, Rajasthan, India
Sensor	Forward-facing DJI camera on a quadcopter
Source resolution	1920×1080 px (video)
Labels	Manual treeline annotations (boxes and masks)
Images	290 in total
Split	255 train / 20 validation / 15 test
Augmentations	
Geometric	Rotation $\pm 15°$
Photometric	Saturation $\pm 25\%$, brightness $\pm 15\%$, exposure $\pm 10\%$
Degradation	Blur up to 1 px, noise up to 0.3% of pixels

Algorithm 1 Greedy NMS vs. Soft-NMS (adapted from [20])

Input: boxes $\mathcal{B} = \{b_1, \ldots, b_N\}$; scores $\mathcal{S} = \{s_1, \ldots, s_N\}$; NMS threshold N_t
Output: kept boxes $\mathcal{D}$ and scores $\mathcal{S}$

$\mathcal{D} \leftarrow \emptyset$
while $\mathcal{B} \neq \emptyset$ **do**
  $m \leftarrow \arg\max \mathcal{S}$; $\mathcal{M} \leftarrow b_m$
  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{M}$; $\mathcal{B} \leftarrow \mathcal{B} \setminus \mathcal{M}$
  for $b_i \in \mathcal{B}$ **do**
    if greedy NMS **then**
      if $\mathrm{IoU}(\mathcal{M}, b_i) \geq N_t$ **then**
        $\mathcal{B} \leftarrow \mathcal{B} \setminus b_i$; $\mathcal{S} \leftarrow \mathcal{S} \setminus s_i$
    else ▷ *Soft-NMS*
      $s_i \leftarrow s_i \, f(\mathrm{IoU}(\mathcal{M}, b_i))$
return $\mathcal{D}, \mathcal{S}$

NMS, however, it penalizes overlap without considering the overall distribution of proposals.

- *Adaptive NMS* [21] adjusts the suppression threshold to the predicted instance density, raising it where objects occlude one another and lowering it where instances are sparse.
- *NOH-NMS* [22] models nearby objects with a Gaussian distribution to refine suppression and is particularly effective for pedestrian detection.

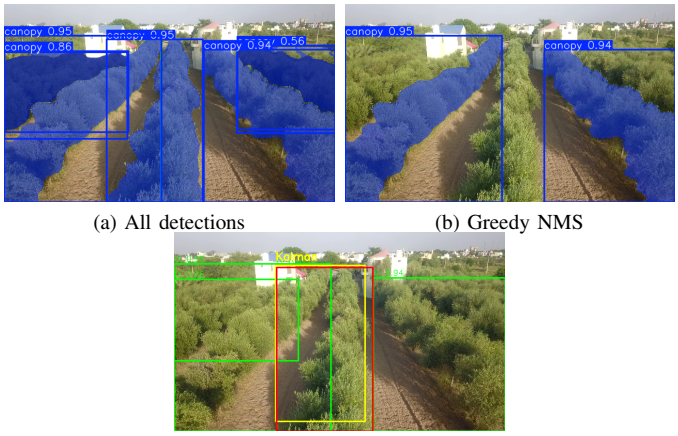
We adopt Soft-NMS with a Gaussian penalty, $\sigma = 0.5$, so that an overlapping detection keeps a reduced score instead of being deleted:

$$s_i' = s_i \, \exp\left(-\frac{\mathrm{IoU}^2(i,j)}{\sigma}\right), \tag{1}$$

where s_i and s_i' are the scores of detection i before and after the update, j is the highest-confidence detection, and σ controls the strength of the suppression. Compared with the linear penalty, the Gaussian form preserves segmentation integrity: overlapping treelines remain separate detections, and closely spaced rows are not merged excessively (Fig. 3c).

C. Target Treeline Lock with a Kalman Filter

After filtering, one treeline is tracked from frame to frame with a Kalman filter [23]. At each frame the filter predicts the



(a) All detections (b) Greedy NMS

(c) Gaussian Soft-NMS + Kalman lock

Fig. 3. Effect of the suppression strategy on converging treelines. (a) All detections with segmentation masks at confidence 0.25. (b) Greedy NMS at an IoU threshold of 0.3 drops individual rows. (c) Gaussian Soft-NMS ($\sigma = 0.5$) keeps them. Green: filtered detections; yellow: Kalman estimate of the target box; red: the detection closest to that estimate.

target box $\hat{\mathcal{B}}_t$, which is matched to the closest detection,

$$i^* = \arg\min_i \left\| \mathcal{B}_i - \hat{\mathcal{B}}_t \right\|, \tag{2}$$

and the two are fused before the filter update:

$$\mathcal{B}_t = 0.65\, \hat{\mathcal{B}}_t + 0.25\, \mathcal{B}_{i^*}. \tag{3}$$

The prediction receives the larger weight because orchard trees are usually planted in straight lines. This choice suppresses brief disturbances, such as birds flying near the camera. The detection term lets the tracker follow genuine changes in real time, such as drift away from a straight trajectory in windy conditions.

D. Mask Refinement and Centerline Extraction

The mask $\mathcal{M}_{i^*}$ of the matched detection can still contain fragments of neighboring treelines or other vegetation, such as shrubs and bushes. Small fragments are removed by a morphological opening with a 5×5 elliptical kernel $K_{5\times5}$, and the largest connected component is kept as the treeline:

$$\mathcal{M}^* = \mathrm{LCC}\big(\mathcal{M}_{i^*} \circ K_{5\times5}\big), \tag{4}$$

where $\circ$ denotes opening and LCC the largest connected component. For every image row y_j that contains mask pixels, the centerline point is the midpoint of the leftmost and rightmost mask pixels:

$$x_j = \tfrac{1}{2}\left(x_j^{\mathrm{left}} + x_j^{\mathrm{right}}\right), \qquad j = 1, \ldots, M. \tag{5}$$

E. Navigation-Line Estimation

Four estimators of the navigation line $\mathcal{L}$ were evaluated (Fig. 4):

(a) a straight line fitted to the entire segmented region;
(b) a straight line $y = mx + c$ fitted through the center points $\{(x_j, y_j)\}$;

Algorithm 2 UAV Treeline Tracking and Navigation

Input: video stream I_t
Output: attitude commands $(\bar{\psi}_t, \bar{\phi}_t, \theta_{\text{const}})$

initialize YOLOv11 model $\mathcal{N}$, $\sigma \leftarrow 0.5$, Kalman filter $\mathcal{K}$, $\alpha \leftarrow 0.35$
for $t = 1, 2, \ldots$ **do**
  $\{\mathcal{B}_i, \mathcal{S}_i, \mathcal{M}_i\}_{i=1}^{N} \leftarrow \mathcal{N}(I_t)$ ▷ *segment*
  $\{\mathcal{B}_i, \mathcal{S}_i, \mathcal{M}_i\} \leftarrow \text{SoftNMS}(\sigma)$ ▷ *Eq.* (1)
  $\hat{\mathcal{B}}_t \leftarrow \mathcal{K}.\text{predict}()$
  $i^* \leftarrow \arg\min_i \|\mathcal{B}_i - \hat{\mathcal{B}}_t\|$ ▷ *match*
  $\mathcal{B}_t \leftarrow 0.65\,\hat{\mathcal{B}}_t + 0.25\,\mathcal{B}_{i^*}$; $\mathcal{K}.\text{update}(\mathcal{B}_t)$ ▷ *fuse*
  $\mathcal{M}' \leftarrow \text{Opening}(\mathcal{M}_{i^*}, K_{5\times5})$ ▷ *clean*
  $\mathcal{M}^* \leftarrow$ largest connected component of $\mathcal{M}'$
  $\{(x_j, y_j)\}_{j=1}^{M} \leftarrow \text{CenterPoints}(\mathcal{M}^*)$
  $\mathcal{L} : y = mx + c \leftarrow \text{LineFit}(\{(x_j, y_j)\})$
  $\Delta x \leftarrow (x_{\text{img}} - x_{\mathcal{L}})/2$; $\phi_t \leftarrow \Delta x \cdot \text{FOV}_x / W$
  $\psi_t \leftarrow \tan^{-1}(1/m) \cdot 180/\pi$
  $\bar{\phi}_t \leftarrow \alpha\phi_t + (1-\alpha)\bar{\phi}_{t-1}$; $\bar{\psi}_t \leftarrow \alpha\psi_t + (1-\alpha)\bar{\psi}_{t-1}$
  send $(\bar{\psi}_t, \bar{\phi}_t, \theta_{\text{const}})$ to PX4

(c) a cubic B-spline fitted to the center points with a smoothness parameter of 500;
(d) a quadratic B-spline fitted to the center points with a smoothness parameter of 500.

For the spline estimators, the slope m is that of the tangent at the curve's center, evaluated at the same vertical coordinate as the image center. The smoothness parameter was chosen empirically through systematic testing. For further stability, the navigation line passes through a second smoothing stage after the Kalman filter: exponential smoothing with factor $\alpha = 0.35$.

F. Control Law

The selected navigation line is converted into attitude commands. Its horizontal offset from the image center is

$$\Delta x = \frac{x_{\text{img}} - x_{\mathcal{L}}}{2}, \tag{6}$$

where $x_{\mathcal{L}}$ is the x-coordinate of the navigation-line center. The roll angle, which sets the lateral tilt, scales this offset by the horizontal field of view FOV_x:

$$\phi = \frac{\Delta x \cdot \text{FOV}_x}{W}. \tag{7}$$

The yaw angle, which corrects the heading, follows from the slope m of the line:

$$\psi = \tan^{-1}\left(\frac{1}{m}\right) \cdot \frac{180}{\pi}. \tag{8}$$

Both angles are smoothed exponentially to avoid abrupt control changes,

$$\bar{\psi}_t = \alpha\,\psi_t + (1-\alpha)\,\bar{\psi}_{t-1}, \qquad \bar{\phi}_t = \alpha\,\phi_t + (1-\alpha)\,\bar{\phi}_{t-1}, \tag{9}$$

where the smoothing factor α sets how much past values influence the updated angles. Pitch is held at a constant θ_{const}. Algorithm 2 collects the complete per-frame procedure.

TABLE IV
INSTANCE-SEGMENTATION ACCURACY ON THE CUSTOM DATASET

	Box mAP (%)		Mask mAP (%)	
Model	@50	@50–95	@50	@50–95
YOLACT++ [24]	69.69	42.58	72.63	54.36
YolactEdge [25]	73.17	46.52	75.06	59.51
YOLOv11 [19]	**95.5**	**78.5**	**96.5**	**71.4**
Gain over YolactEdge	*+22.33*	*+31.98*	*+21.44*	*+11.89*

VI. EXPERIMENTS AND RESULTS

A. Segmentation Accuracy

YOLOv11 was compared with two real-time instance-segmentation models, YOLACT++ [24] and YolactEdge [25], all trained on the dataset of Section IV. Performance is reported as box and mask mean average precision (mAP) at an IoU threshold of 0.50 (mAP@50) and averaged over thresholds from 0.50 to 0.95 (mAP@50–95). Table IV gives the exact values, and Fig. 5 compares them visually.

Box accuracy. YOLOv11 achieves the highest box mAP, 95.5 at mAP@50 and 78.5 at mAP@50–95, well above YolactEdge (73.17 and 46.52) and YOLACT++ (69.69 and 42.58). The largest gap appears at the stricter mAP@50–95 metric, which indicates that YOLOv11 localizes treeline boundaries more tightly, not merely more often. We attribute this to its anchor-free detection head and attention-based feature extraction.

Mask accuracy. YOLOv11 also leads in mask mAP, with 96.5 at mAP@50 and 71.4 at mAP@50–95, compared with 75.06 and 59.51 for YolactEdge and 72.63 and 54.36 for YOLACT++. It produces each instance mask directly in a single end-to-end pass. YOLACT-family models instead assemble masks from shared prototypes, combined with per-instance coefficients and cropped by the predicted box, so errors in the box propagate into the mask. Of the two YOLACT variants, YolactEdge performs better, which is consistent with its improved feature pyramid network (FPN) and GPU-optimized architecture.

B. Effect of Soft-NMS

Fig. 3 illustrates the failure that motivated the overlap-aware front end. With greedy NMS at an IoU threshold of 0.3, a box spanning several converging rows suppresses the boxes of the individual rows (Fig. 3b). Gaussian Soft-NMS retains them with reduced scores, so the Kalman filter can still associate the target row with its own detection (Fig. 3c).

C. Navigation-Line Selection

Table V summarizes the behavior of the four estimators across extensive testing (Fig. 4). The line fitted to the whole region (a) was highly sensitive to changes in the detected mask and therefore unreliable. The line through the center points (b) was more stable and accurate, gave consistent yaw estimates, and was adopted for navigation. The cubic (c) and quadratic (d) splines were similarly sensitive, because their tangent depends on the shape of the treeline and on the morphological operations. The cubic tangent was also steeper than the quadratic one,

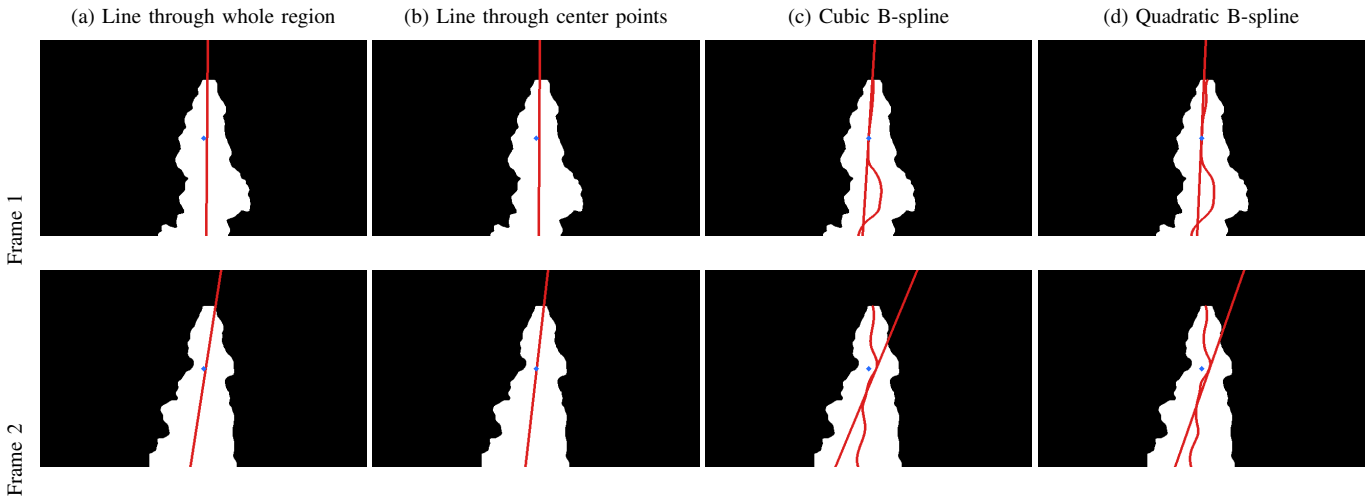


Fig. 4. The four navigation-line estimators applied to the refined treeline mask (white) in two frames. In (c) and (d), the straight line is the tangent at the spline center. The blue dot marks the image center. Line widths are thickened for legibility.

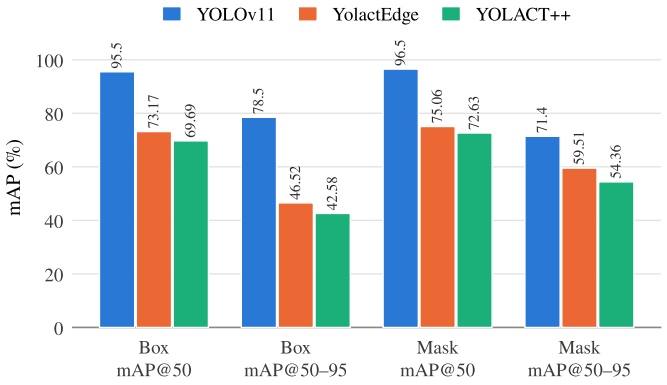


Fig. 5. Box and mask mAP of the three instance-segmentation models on the custom dataset (values from Table IV).

TABLE V
QUALITATIVE COMPARISON OF NAVIGATION-LINE ESTIMATORS

Estimator	Yaw behavior	Roll	Used
(a) Line, whole region	Highly sensitive to mask changes	Stable	–
(b) Line, center points	Stable, consistent yaw	Stable	✓
(c) Cubic B-spline	Tangent shape-sensitive; steepest tangent	Stable	–
(d) Quadratic B-spline	Tangent shape-sensitive	Stable	–

which reduced stability further. The roll angle stayed relatively constant for all estimators because the x-offset was consistent in most cases.

D. Onboard Runtime

Based on its accuracy, YOLOv11 was deployed on the NVIDIA Jetson Orin NX with TensorRT INT16 optimization. Table VI lists the measured timings. The pipeline runs in real time at 17 FPS, which makes it suitable for edge deployment where fast and accurate instance segmentation is required.

TABLE VI
ONBOARD RUNTIME ON NVIDIA JETSON ORIN NX (TENSORRT)

Stage	Time
YOLOv11 inference	43.48 ms
Post-processing (Soft-NMS, morphology, navigation line)	58.82 ms
Reported real-time throughput	17 FPS

E. Field Trials

The complete system was flown on a quadcopter in the olive orchards of Pilani. The UAV generated and followed navigation lines autonomously, which allowed efficient scanning of the orchard. The integrated real-time tracking kept the quadcopter stable even under small wind disturbances.

VII. DISCUSSION AND LIMITATIONS

Two design choices drive the system's robustness. Flying above the canopy turns row following into a segmentation problem on large, high-contrast structures. Trunks and the inter-row path, which are easily occluded, are not needed. Gaussian Soft-NMS then removes the main failure that above-canopy viewing introduces, the mutual suppression of converging rows. Among the navigation-line estimators, the simplest robust one (a straight line through the center points) proved better than the more flexible splines. Spline tangents respond to local irregularities in the mask that do not reflect the row's true heading.

The current study has limitations that point to future work. The flight evaluation is qualitative: quantitative tracking metrics, such as cross-track error, heading error, and track-loss rate under measured wind, have yet to be reported. The dataset covers a single olive orchard and contains 15 test images, so generalization to other crops, growth stages, and lighting conditions remains to be established. Finally, the Kalman fusion weights, smoothing factor, and spline smoothness were set empirically; a sensitivity analysis would clarify how they affect stability.

VIII. Conclusion

This paper presented a monocular, onboard method for tracking orchard treelines from a low-altitude UAV. It combines YOLOv11 canopy instance segmentation, Gaussian Soft-NMS, Kalman-based target locking, and a center-point navigation line that maps directly to roll and yaw commands at constant pitch. On a new olive-orchard dataset, YOLOv11 clearly outperformed YolactEdge and YOLACT++, with a box mAP@50 of 95.5% and a mask mAP@50 of 96.5%. The full pipeline ran at 17 FPS on a Jetson Orin NX and followed treelines autonomously in field trials, remaining stable under small wind disturbances. Future work will explore other Soft-NMS variants, improve navigation-line generation, and optimize orchard scanning for fully automated coverage, with provision for manual intervention when needed.

Acknowledgment

This work was supported by CSIR-CEERI, Pilani.

References

[1] E. Roma, P. Catania, M. Vallone, and S. Orlando, "Assessing the effectiveness of pruning in an olive orchard using a drone and a multispectral camera: A three-year study," *Agronomy*, vol. 14, no. 5, Art. no. 1023, 2024, doi: 10.3390/agronomy14051023.

[2] J. Li *et al.*, "A multi-area task path-planning algorithm for agricultural drones based on improved double deep Q-learning net," *Agriculture*, vol. 14, no. 8, Art. no. 1294, 2024, doi: 10.3390/agriculture14081294.

[3] E. V. Vazquez-Carmona, J. I. Vasquez-Gomez, J. C. Herrera-Lozada, and M. Antonio-Cruz, "Coverage path planning for spraying drones," *Comput. Ind. Eng.*, vol. 168, Art. no. 108125, 2022, doi: 10.1016/j.cie.2022.108125.

[4] A. Basiri, V. Mariani, G. Silano, M. Aatif, L. Iannelli, and L. Glielmo, "A survey on the application of path-planning algorithms for multi-rotor UAVs in precision agriculture," *J. Navig.*, vol. 75, no. 2, pp. 364–383, 2022, doi: 10.1017/S0373463321000825.

[5] R. I. Mukhamediev *et al.*, "Coverage path planning optimization of heterogeneous UAVs group for precision agriculture," *IEEE Access*, vol. 11, pp. 5789–5803, 2023, doi: 10.1109/ACCESS.2023.3235207.

[6] R. Tian, M. Cao, F. Ma, and P. Ji, "Agricultural UAV path planning based on improved A* and gravity search mixed algorithm," *J. Phys.: Conf. Ser.*, vol. 1631, Art. no. 012082, 2020, doi: 10.1088/1742-6596/1631/1/012082.

[7] J. Radcliffe, J. Cox, and D. M. Bulanon, "Machine vision for orchard navigation," *Comput. Ind.*, vol. 98, pp. 165–171, 2018, doi: 10.1016/j.compind.2018.03.008.

[8] J. Feng, G. Liu, Y. Si, S. Wang, B. He, and W. Ren, "Algorithm based on image processing technology to generate navigation directrix in orchard," *Trans. Chin. Soc. Agric. Mach.*, vol. 43, no. 7, pp. 184–189, 2012, doi: 10.6041/j.issn.1000-1298.2012.07.034.

[9] B. He, G. Liu, Y. Ji, Y. Si, and R. Gao, "Auto recognition of navigation path for harvest robot based on machine vision," in *Computer and Computing Technologies in Agriculture IV* (IFIP Adv. Inf. Commun. Technol.). Berlin, Germany: Springer, 2011, pp. 138–148, doi: 10.1007/978-3-642-18333-1_19.

[10] W. Ali, F. Georgsson, and T. Hellström, "Visual tree detection for autonomous navigation in forest environment," in *Proc. IEEE Intell. Vehicles Symp.*, Eindhoven, The Netherlands, 2008, pp. 560–565, doi: 10.1109/IVS.2008.4621315.

[11] A. S. Brandão, F. N. Martins, and H. B. Soneguetti, "A vision-based line following strategy for an autonomous UAV," in *Proc. 12th Int. Conf. Informatics Control, Automat. Robot. (ICINCO)*, vol. 2, Colmar, France, 2015, pp. 314–319, doi: 10.5220/0005543903140319.

[12] N. Stefas, H. Bayram, and V. Isler, "Vision-based monitoring of orchards with UAVs," *Comput. Electron. Agric.*, vol. 163, Art. no. 104814, 2019, doi: 10.1016/j.compag.2019.05.023.

[13] P. M. Blok, K. van Boheemen, F. K. van Evert, J. IJsselmuiden, and G.-H. Kim, "Robot navigation in orchards with localization based on particle filter and Kalman filter," *Comput. Electron. Agric.*, vol. 157, pp. 261–269, 2019, doi: 10.1016/j.compag.2018.12.046.

[14] D. Popescu, L. Ichim, and F. Stoican, "Orchard monitoring based on unmanned aerial vehicles and image processing by artificial neural networks: A systematic review," *Front. Plant Sci.*, vol. 14, Art. no. 1237695, 2023, doi: 10.3389/fpls.2023.1237695.

[15] J. Zhou, S. Geng, Q. Qiu, Y. Shao, and M. Zhang, "A deep-learning extraction method for orchard visual navigation lines," *Agriculture*, vol. 12, no. 10, Art. no. 1650, 2022, doi: 10.3390/agriculture12101650.

[16] Z. Han, J. Li, Y. Yuan, X. Fang, B. Zhao, and L. Zhu, "Path recognition of orchard visual navigation based on U-Net," *Trans. Chin. Soc. Agric. Mach.*, vol. 52, no. 1, pp. 30–39, 2021, doi: 10.6041/j.issn.1000-1298.2021.01.004.

[17] S. Back, G. Cho, J. Oh, X.-T. Tran, and H. Oh, "Autonomous UAV trail navigation with obstacle avoidance using deep neural networks," *J. Intell. Robot. Syst.*, vol. 100, no. 3–4, pp. 1195–1211, 2020, doi: 10.1007/s10846-020-01254-5.

[18] N. Smolyanskiy, A. Kamenev, J. Smith, and S. Birchfield, "Toward low-flying autonomous MAV trail navigation using deep neural networks for environmental awareness," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Vancouver, BC, Canada, 2017, pp. 4241–4247, doi: 10.1109/IROS.2017.8206285.

[19] R. Khanam and M. Hussain, "YOLOv11: An overview of the key architectural enhancements," 2024, *arXiv:2410.17725*.

[20] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, "Soft-NMS—Improving object detection with one line of code," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, 2017, pp. 5562–5570, doi: 10.1109/ICCV.2017.593.

[21] S. Liu, D. Huang, and Y. Wang, "Adaptive NMS: Refining pedestrian detection in a crowd," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, 2019, pp. 6452–6461, doi: 10.1109/CVPR.2019.00662.

[22] P. Zhou *et al.*, "NOH-NMS: Improving pedestrian detection by nearby objects hallucination," in *Proc. 28th ACM Int. Conf. Multimedia*, Seattle, WA, USA, 2020, pp. 1967–1975, doi: 10.1145/3394171.3413617.

[23] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.

[24] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, "YOLACT++: Better real-time instance segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 2, pp. 1108–1121, 2022, doi: 10.1109/TPAMI.2020.3014297.

[25] H. Liu, R. A. Rivera Soto, F. Xiao, and Y. J. Lee, "YolactEdge: Real-time instance segmentation on the edge," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, Xi'an, China, 2021, pp. 9579–9585, doi: 10.1109/ICRA48506.2021.9561858.